

SALUS SECURITY

MAY 2026



CODE SECURITY ASSESSMENT

PEAK AI

Overview

Project Summary

- Name: PeakAI
- Platform: EVM-compatible chains
- Language: Solidity
- Repository:
 - <https://github.com/WSbaikaishui/peak>
- Audit Range: See [Appendix - 1](#)

Project Dashboard

Application Summary

Name	PeakAI
Version	v2
Type	Solidity
Dates	May 18 2026
Logs	May 13 2026, May 18 2026

Vulnerability Summary

Total High-Severity issues	0
Total Medium-Severity issues	1
Total Low-Severity issues	1
Total informational issues	2
Total	4

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction	4
1.1 About SALUS	4
1.2 Audit Breakdown	4
1.3 Disclaimer	4
Findings	5
2.1 Summary of Findings	5
2.2 Notable Findings	6
1. Centralization risk	6
2. Possible incorrect remaining mintable calculation	7
2.3 Informational Findings	8
3. Mapping Declaration Lacks Named Parameters for Improved Readability	8
4. Lack of Security Contact Information for Responsible Disclosure	9
Appendix	10
Appendix 1 - Files in Scope	10

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (<https://t.me/salusec>), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Centralization risk	Medium	Centralization	Acknowledged
2	Possible incorrect remaining mintable calculation	Low	Business Logic	Resolved
3	Mapping Declaration Lacks Named Parameters for Improved Readability	Informational	Code Quality	Resolved
4	Lack of Security Contract Information for Responsible Disclosure	Informational	Configuration	Acknowledged

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Centralization risk	
Severity: Medium	Category: Centralization
Target: - src/PeakAI.sol	

Description

In the PeakAI contract, there exists one privileged role, `owner`. This role has the authority to execute some key functions such as `addMinter`.

If this role's private keys are compromised, an attacker could trigger these functions to mint all tokens.

src/PeakAI.sol:L49-L55

```
function addMinter(address account) external onlyOwner {
    require(account != address(0), "ERC20: cannot add zero address");
    require(!_minterWhitelist.contains(account), "ERC20: account already in whitelist");
    require(_minterWhitelist.length() < MAX_MINTERS, "ERC20: max minters reached");
    _minterWhitelist.add(account);
    emit MinterAdded(account);
}
```

Recommendation

We recommend transferring privileged accounts to multi-sig accounts with timelock governors for enhanced security. This ensures that no single person has full control over the accounts and that any changes must be authorized by multiple parties.

Status

This issue has been acknowledged by the team.

2. Possible incorrect remaining mintable calculation

Severity: Low

Category: Business Logic

Target:

- src/PeakAI.sol

Description

The function `remainingMintableForAddress` is used to calculate the remaining mintable token amount.

If one account is not in the whitelist or removed from the whitelist, the function will return an incorrect value.

src/PeakAI.sol:L123-L128

```
function remainingMintableForAddress(address account) public view returns (uint256) {  
    if (mintedAmounts[account] >= maxMintPerAddress) {  
        return 0;  
    }  
    return maxMintPerAddress - mintedAmounts[account];  
}
```

Recommendation

If one account is not in the whitelist, return 0.

Status

This issue has been resolved by the team with commit [1b5d959](#).

2.3 Informational Findings

3. Mapping Declaration Lacks Named Parameters for Improved Readability

Severity: Informational

Category: Code Quality

Target:

- src/PeakAI.sol

Description

Since Solidity version 0.8.18, the language allows developers to add named key and value identifiers in mapping declarations using the syntax:

```
mapping(KeyType keyName => ValueType valueName)
```

This enhances code readability and clarifies how a mapping is intended to be used. In the PeakAI contract, the mapping `mintedAmounts` is declared without named parameters, making the purpose of the key and value less obvious upon inspection.

src/PeakAI.sol:L23-L26

```
contract PeakAI is ERC20, Ownable2Step {
    EnumerableSet.AddressSet private _minterWhitelist;
    mapping(address => uint256) public mintedAmounts;
    uint256 public maxMintPerAddress;
    ...
}
```

Recommendation

Consider updating the mapping declaration to explicitly name the key and value identifiers, enhancing readability and making the mapping's purpose clearer.

Status

This issue has been resolved by the team with commit [1b5d959](#).

4. Lack of Security Contact Information for Responsible Disclosure

Severity: Informational

Category: Configuration

Target:

- src/PeakAI.sol

Description

The contract PeakAI does not specify a security contact point. Including a designated security contact (such as an email address or ENS name) in the contract's NatSpec header facilitates responsible vulnerability disclosure. This makes it easier for external researchers to quickly reach the appropriate team in the event a vulnerability is identified, helping minimize the time window between discovery and mitigation. The Ethereum community has begun standardizing this practice using the `@custom:security-contact` tag, adopted by tools such as OpenZeppelin Wizard and ethereum-lists.

Recommendation

Consider adding a NatSpec comment at the top of the contract with a `@custom:security-contact` field pointing to the preferred disclosure channel.

Status

This issue has been acknowledged by the team.

Appendix

Appendix 1 - Files in Scope

This audit covered the following files in commit [bdcd419](#):

File	SHA-1 hash
PeakAI.sol	f3b4df26d56b9c3556088df80854a214633e8ca3